



[TECHNICAL WHITEPAPER]

Autonomy Harness

A reactive and proactive cognition stack: autonomous agents that run live operations — on infrastructure you own.

VERSION 1.0 · JULY 2026

Abstract

Persistent autonomy in contested and disconnected operations imposes three requirements that current agent architectures rarely meet together: cognition that outlives any context window, governance that holds even when the model is wrong, and evidence that an independent reviewer can replay. This paper presents the **Autonomy Harness**, a self-hosted cognition stack that meets all three on infrastructure the operator owns. Agent context is a **world model** — everything the operation senses, remembers, and decides, fused into one durable, queryable model of reality — from which every reasoning episode assembles a fresh slice. Enforcement lives outside the agent's reasoning: every action clears one authenticated gateway and answers to installation policy, refused by default and audited always. Long-running work executes as durable tasks with declared recovery semantics that survive process death, and every signal the operation produces is persisted before anything else touches it — losslessly, at approximately 225,000 messages per second. A live traffic-world case study closes the perceive–decide–act loop end to end, and the same platform deploys unchanged from a single edge host to an air-gapped cluster.

KEYWORDS autonomous agents · world model · runtime governance · durable execution · persist-first recording · edge and air-gap deployment · MCP

[READING GUIDE]

Sections 1–2 state the problem and the contributions. Sections 3–9 develop the architecture: the system model, the world model, the agent kernel, governance, the capability plane, durable work, and the record. Section 10 validates the loop on a live traffic world; Sections 11–12 cover deployment and conclusions. Appendix A defines every component.



Contents

#	Section
1	Introduction
2	Contributions
3	System Model
4	The World Model
5	The Agent Kernel
6	Governance: Edge, Access, and the Gateway
7	The Capability Plane
8	Durable Work and Shared Planes
9	The Record
10	Empirical Validation
11	Deployment Considerations
12	Conclusion
A	Appendix: Component Reference



1. Introduction

Autonomous systems are leaving the laboratory and entering operations: traffic networks, sensor deployments, logistics, field robotics. The intelligence available to run them — large language models and other general reasoners — is powerful, but three properties of real operations expose the gap between a capable model and a deployable autonomous system.

Operations outlast context windows. A mission runs for days or weeks; a model's context holds one conversation. An agent whose knowledge lives in its context forgets its mission at every restart, every upgrade, and every machine failure. Persistent autonomy requires memory that is architectural, not conversational.

Models cannot be their own governors. A reasoner can hallucinate, drift over a long mission, or absorb hostile instructions from the environment it observes. Any property that must hold — who may act, on which data, within which bounds — has to be enforced outside the model's own reasoning, and every enforcement decision has to leave evidence.

Operations cannot depend on someone else's cloud. The environments that most need autonomy — contested, disconnected, regulated — are precisely the ones where a vendor control plane, a required hosted service, or a mandatory network link is a liability. The platform must deploy where the operation is, including behind an air gap, and be owned entirely by its operator.

The Autonomy Harness is a cognition stack built for these constraints. Its thesis: give agents a **world model** instead of a chat history, govern them from **outside their own reasoning**, make every piece of long-running work **durable with declared recovery**, persist **every signal before anything else touches it**, and package the whole stack so that one operator can run it anywhere. The remainder of this paper develops each of these commitments and validates them on a live traffic world.



2. Contributions

The Autonomy Harness makes six architectural contributions. Each is developed in the section noted and exercised end to end by the validation in Section 10.

- 1 **The world model as agent context (§4).** Everything the operation senses, remembers, and decides fuses into one durable, queryable model of reality. Every reasoning episode assembles a fresh slice of it, so an agent's knowledge survives any restart and outlives any context window.
- 2 **Cognition governed from outside the model (§6).** Every action clears one authenticated gateway and answers to installation policy — refused by default when in doubt. A model can be wrong, drift, or swallow hostile instructions and still cannot act beyond the envelope; even refusals leave evidence.
- 3 **Persist-first recording (§9).** The ingest socket and the durable writer are one process: every message is written before anything else happens to it. Live files stay readable mid-write, segments are verified before replacement, and capture is lossless at approximately 225,000 messages per second.
- 4 **Durable tasks with declared recovery semantics (§8).** Long work detaches from the episode that started it and survives process death. Every operation declares its recovery class — resumable, webhook-bound, or fail-closed — and completion wakes the sleeping agent within a second.
- 5 **A modular capability plane under one contract (§7).** Nine capability modules run at the edge today; any MCP server — local, bridged, or remote — registers in a validated catalog, applies live, and immediately answers to the same task, artifact, and policy contract.
- 6 **Deployment-invariant cognition (§11).** The same platform, the same agents, and the same operational intelligence run on one edge host, across a cluster, sealed inside an air gap, or hybrid — with provenance included and connectivity on the operator's terms.



3. System Model

The system closes one loop. The **world** — sensors, simulators, and the agents themselves — streams into the **record**; agents perceive from it, decide, and act through **governed tools**; their effects land back in the world; and every step deposits **evidence**. Three flows share this spine. **Observe**: producers push into the recording hub, and agents query the record back out through governed tools. **Act**: agents and operators call tools through the gateway; long work becomes a durable task whose completion wakes the sleeping agent. **Account**: every authentication decision, tool call, task transition, artifact grant, and agent step lands in the platform store and the decision log.

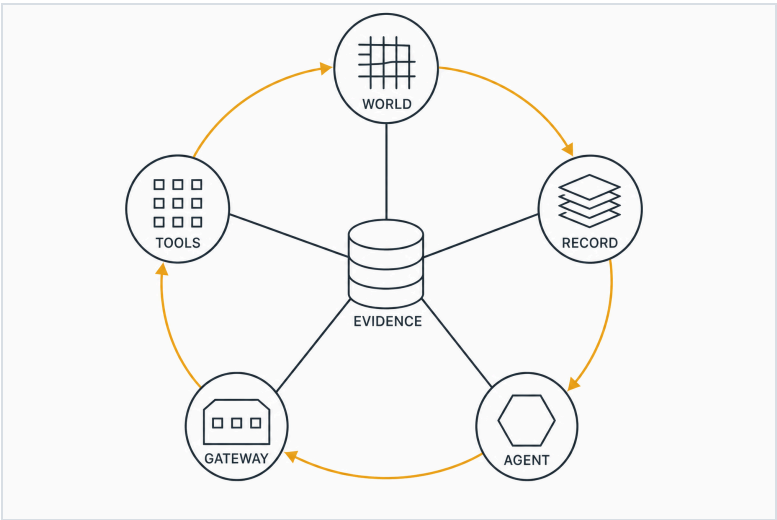


FIGURE 1 – THE AUTONOMY LOOP: THE WORLD IS RECORDED, AGENTS PERCEIVE AND ACT THROUGH GOVERNED TOOLS, EFFECTS LAND BACK IN THE WORLD, AND EVERY STEP LEAVES EVIDENCE.

Flow	Path	Guarantee
Observe	Producers push typed streams to one ingest point; segments become governed catalog records; agents read slices through the recording module.	The hub persists first; a crash mid-write leaves readable files.
Act	Every tool call enters through the gateway under one flat namespace; quick tools answer inline; long tools return a durable task handle immediately.	Task completion pushes a wake; the next episode assembles with the result.
Account	Auth, policy, admin, and task transitions write audit evidence; every decision records what the agent saw, chose, and why.	Usage records meter provider spend and tool activity per principal.



4. The World Model

Agent context is a **world model**: everything the operation senses, remembers, and decides, fused into one durable, queryable model of reality. Sensors and simulators stream the physical world into the record; agents keep their beliefs, missions, and constraints in analytical memory; every decision lands in the log — and each act an agent takes becomes part of the model the next episode reasons from. Context is assembled fresh from the world model every episode, so an agent's knowledge survives any restart and outlives any context window.

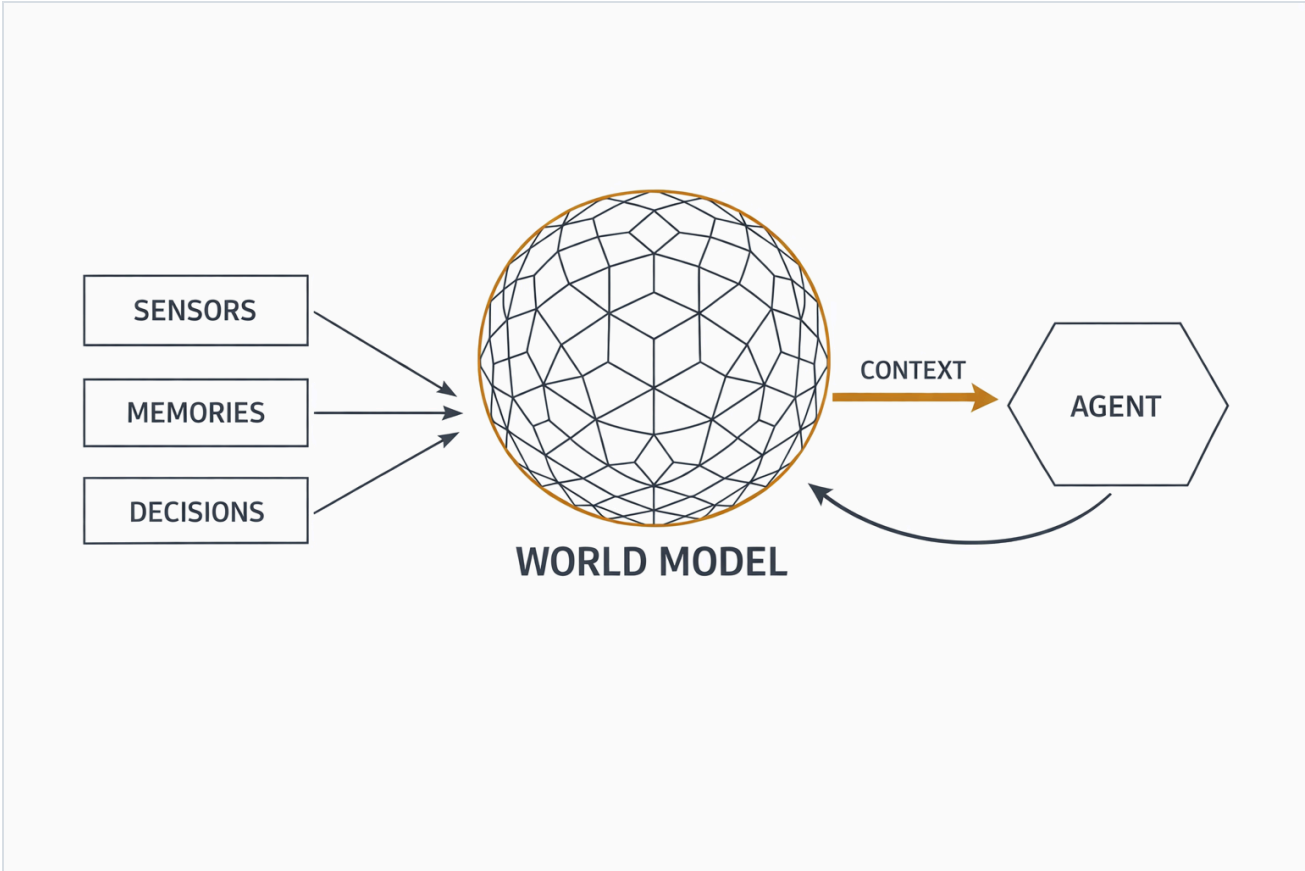


FIGURE 2 – SENSORS, MEMORIES, AND DECISIONS FUSE INTO THE WORLD MODEL; EVERY EPISODE'S CONTEXT IS A FRESH SLICE OF IT.

Source	What it contributes	What agents draw
Sensors & sims	The physical world, streamed into the durable time-and-space record.	The world now (latest-at) · how it got here (trajectories and ranges).
Agent memory	Beliefs, missions, constraints, and state in each agent's analytical memory.	What the agent believes and intends — queryable with guarded SQL.
Decision log	Every turn, tool call, and rationale, time-indexed and replayable.	What happened and why — the model of the agents themselves.



5. The Agent Kernel

An agent runs indefinitely because durable stores are the memory: every episode's context is assembled fresh from the world model, and everything in flight survives process death. The kernel executes typed manifests — an agent type is a manifest, a preamble, and SQL migrations — so fielding another agent is an act of configuration, and one installation runs one agent or a team of them.

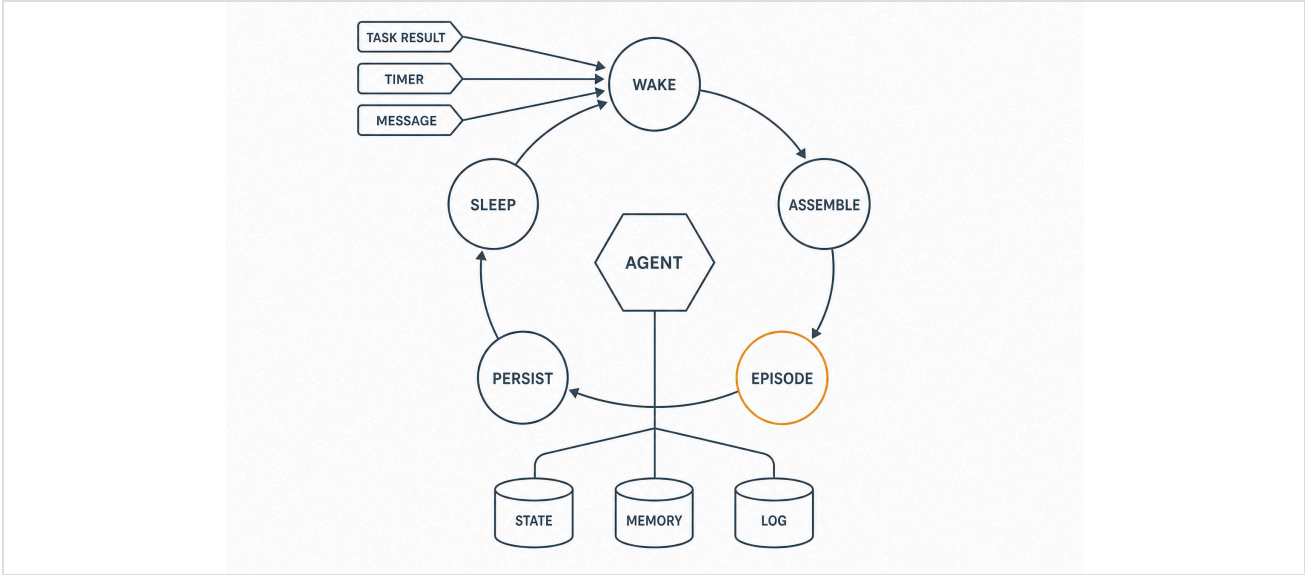


FIGURE 3 – THE LIFECYCLE: WAKE → ASSEMBLE → EPISODE → PERSIST → SLEEP, GROUNDED ON THE DURABLE PLANES.

Stage	What happens
Wake	A batch of wakes, coalesced and deduped; every wake is persisted with its disposition, so the ledger explains why each episode ran.
Assemble	A fresh slice of the world model — current state, beliefs, and history — plus the wake payload, under a token budget.
Episode	One bounded reasoning run. Quick gateway tools answer inline; long tools dispatch as durable tasks. Hooks record every step and terminate over-budget runs.
Persist	Unresolved task descriptors land in the platform store; the episode closes with usage counters and a summary.
Sleep	Idle on the wake bus. A leased watcher per detached task waits on the result; the heartbeat bounds silence.

Wakes: task result · resource change · timer · operator message · elicitation answer. A wake survives process death — descriptors and watcher leases rehydrate from the platform store on any boot, so kill and restart resumes the same tasks.



5.1 Memory Planes and Durability

Memory planes

- **Scheduling truth.** The platform store owns episodes, wakes, task watchers, and leases; the transactional outbox and changefeed deliver wakes across processes.
- **Analytical memory.** A sandboxed analytical database per agent: domain tables (the Pilot: targets, missions, waypoints, constraints, beliefs), a kv store, and an episode-log projection for context assembly.
- **Decision log.** An append-only recording: every turn, tool call, task lifecycle, prompt, and summary — time-indexed and watchable live.
- The agent reads all three itself: `memory_query` (guarded SQL), `memory_write` (typed mutations), and `timeline_query` (dataframe reads over the log).

Durability, measured

- Detach at episode end: a task's descriptor outlives the process; any later boot rehydrates the live handle and keeps waiting.
- Token rotation is make-before-break; task ids are principal-scoped, so continuity holds across reconnects.
- The wake is push, arithmetically: with polling pinned at 60 s and the heartbeat at 600 s, a validation agent slept a task's full ~15 s and woke within one second of completion — verified live with a real model end to end.
- Budget hooks book overruns as `budget_terminated`, a first-class outcome; hourly windows defer non-priority wakes.

[PRINCIPLE]

Agents are data. A typed manifest — model endpoint, gateway principal, episode limits, budgets, schedule, context sections, writable tables — is validated fail-closed at boot. A preamble and domain migrations complete the profile; cross-episode instructions live there or in memory.



6. Governance: Edge, Access, and the Gateway

Every client enters through the installation's own ingress, and every action runs the same gauntlet: authenticate, then policy, then audit. Enforcement lives outside the agent's own reasoning — a model can be wrong, drift, or swallow hostile instructions and still cannot act beyond the envelope the operator set. The only path onward carries a gateway-signed identity, and every edge module lives on the internal network behind it.

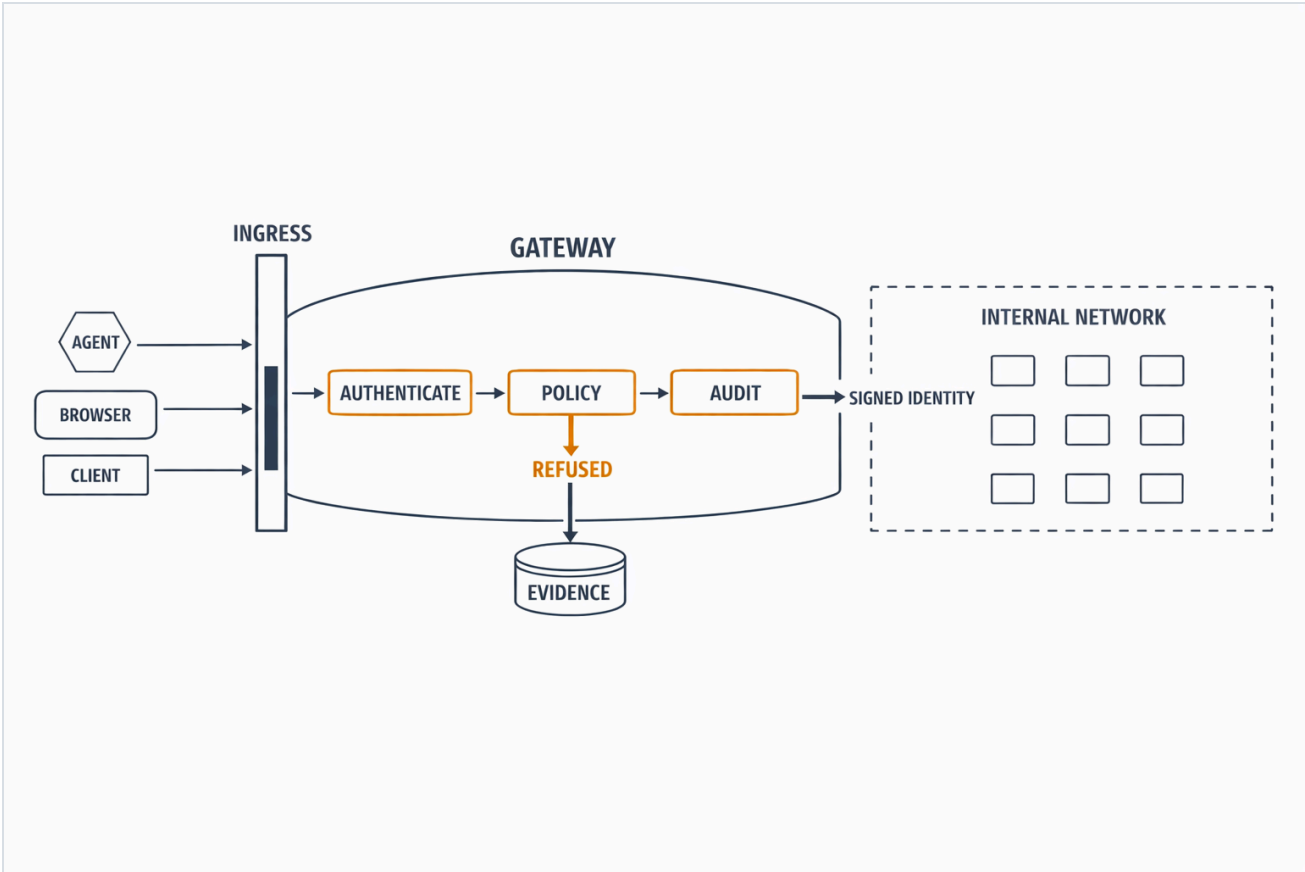


FIGURE 4 — ONE DOOR IN, AUTHENTICATE → POLICY → AUDIT, REFUSED BY DEFAULT, SIGNED IDENTITY ONWARD, EVIDENCE KEPT.

Function	Behavior
Authentication	Browser OAuth with PKCE, client credentials with private-key JWT, identity-assertion grant exchange, JWKS, token issuance and revocation, durable refresh rotation — reuse revokes the family.
Policy & audit	Typed attribute- and role-based policy over tools, resources, tasks, and artifacts; tenant and data-label checks for regulated data; fail-closed decisions with reason codes; a durable audit trail.
Control plane	Validated module / profile / policy catalog, authenticated live apply and reload, secret references with redaction, an admin API for the console, and policy-checked, audited artifact downloads.
Protocol surface	Tools, resources and templates, prompts, completions, tasks, subscriptions, notifications, artifacts, and usage — forwarded and aggregated under one flat tool namespace.



7. The Capability Plane

Capability is modular. Nine modules run at the edge today, inside the installation, behind gateway-signed identity — and the surface is open: a new capability, whether another edge server, a stdio server carried by the bridge, or a remote MCP endpoint, registers in the validated catalog, applies live, and immediately answers to the same tasks, artifacts, and policy contract. The gateway projects every module into one flat tool menu.

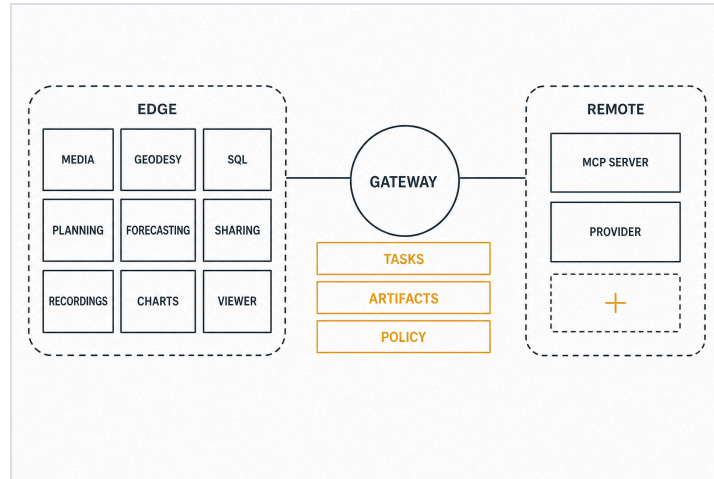


FIGURE 5 — EDGE AND REMOTE CAPABILITY MODULES; ONE CONTRACT GOVERNS ALL, AND THE NEXT SLOT IS OPEN.

Module	Capability	Tools
Media media	Provider-backed generative media with webhook-driven completion, a model catalog, and pricing.	run · models · model_schema · artifact
Geodesy coordinates	Geodesy and reference-frame math: WGS84 / ECEF / ENU / NED, projected CRS, geofences.	convert_frame · transform_crs · derive_local_frame · geodesic_inverse · geodesic_direct · validate_geofence · batch_transform
SQL duckdb	Arbitrary SQL over principal-owned databases inside a locked, resource-bounded sandbox.	query · execute · ingest · export
Planning optimization	Task-option planning for one or many agents over typed objects or SQL-readable option rows.	plan
Forecasting timeseries	Forecasting over any SQL-readable time-series source, visualized as recordings.	forecast
Sharing artifact	Discovery, metadata, grants, release, and sharing — the governance surface over the byte store.	metadata · grant_access · revoke_access · set_release_state · create_share_link · revoke_share_link
Recordings recording	Authorized access to the record: discovery, dataframe queries, subscriptions, publication.	query_recording · seal_recording
Charts charts	Chart rendering projected through the gateway, turning query results into shareable visuals.	render_chart · compile_chart · validate_chart · list_chart_types · create_chart_view
Viewer rerun	A live viewer driven entirely over MCP — session, capture, inspection, and UI automation.	17 interactive tools (connect ... batch)



8. Durable Work and Shared Planes

Durable tasks

Every long-running tool runs on the shared task runtime. Task identities are time-ordered; creation, leases, transitions, results, and outbox events are atomic in the platform store. Recovery is declared per operation, so an interruption always has defined, recorded semantics:

Class	Meaning
<code>resume</code>	Deterministic, side-effect-safe work may be reclaimed after its lease expires.
<code>webhook_wait</code>	A durably submitted provider job waits for its signed webhook. Completion is webhook-only.
<code>interrupted_indeterminate</code>	Interrupted mutating work fails closed and is never replayed automatically.

Shared planes

- **Platform store.** One durable coordination authority for identity, policy, control-plane revisions, tasks, artifacts, recordings, agents, wakes, audit evidence, and usage. The transactional outbox and replayable changefeed carry all cross-process work.
- **Artifact plane.** A single byte-level policy-enforcement point: object storage for bytes, the platform store for grants, labels, and release state. Fresh opaque identity per occurrence; hashes provide integrity and tenant-local dedup; artifact references let one module consume another's output.
- **Contract core.** The typed, provider-neutral contract every module shares, validated fail-closed at startup: strong identities, deployment profiles, shared task and artifact schemas, and gateway-signed identity assertions — modules hold public verification keys only.

[PRINCIPLE]

Two sharing modes, both explicit: authorized grants to users or groups, still constrained by tenant and label policy; and read-only anyone-with-link bearers for artifacts explicitly marked releasable — expiring, download-limited, and revocable.



9. The Record

Every sensor, estimator, and agent streams into one ingest point, and the hub persists first: each message is written — fsynced — to day-partitioned segment files the moment it arrives. Live files stay readable mid-write, so a crash leaves a decodable record, and every segment is verified before its optimized replacement when it freezes. Every recording and segment becomes a governed record in the platform store, and the recordings module answers authorized queries over the same bytes. The record is simultaneously the agents' ground truth and the operation's evidence chain: any run replays, any moment is queryable, and when someone asks why the system acted, the record answers.

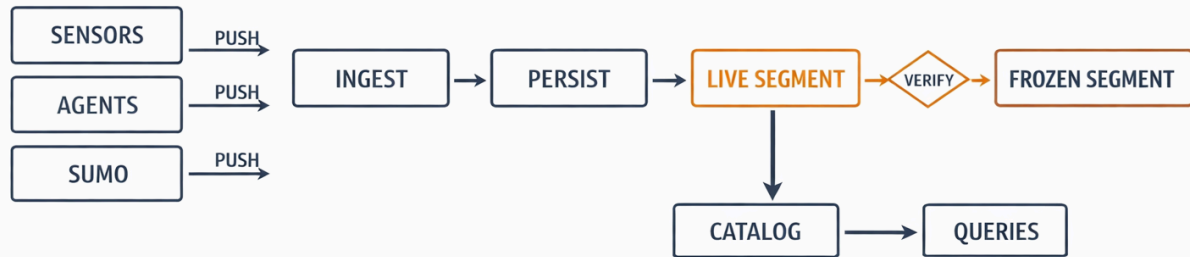


FIGURE 6 — DURABLE CAPTURE: A CRASH LEAVES A READABLE RECORD · A RESTART NEVER OVERWRITES · ~225K MSGS/S LOSSLESS.

10. Empirical Validation

The claims above are exercised on a live traffic world. The SUMO microsimulator runs the pinned, MIT-licensed LuST Luxembourg scenario in its own container; one `sumo-mcp` module owns the single serialized control connection, pushes the world into the record as typed streams, and exposes traffic control as governed tools. The long operations are durable tasks the agent detaches from and wakes on — the same sleep/wake the kernel runs everywhere, now driven by a real simulator.

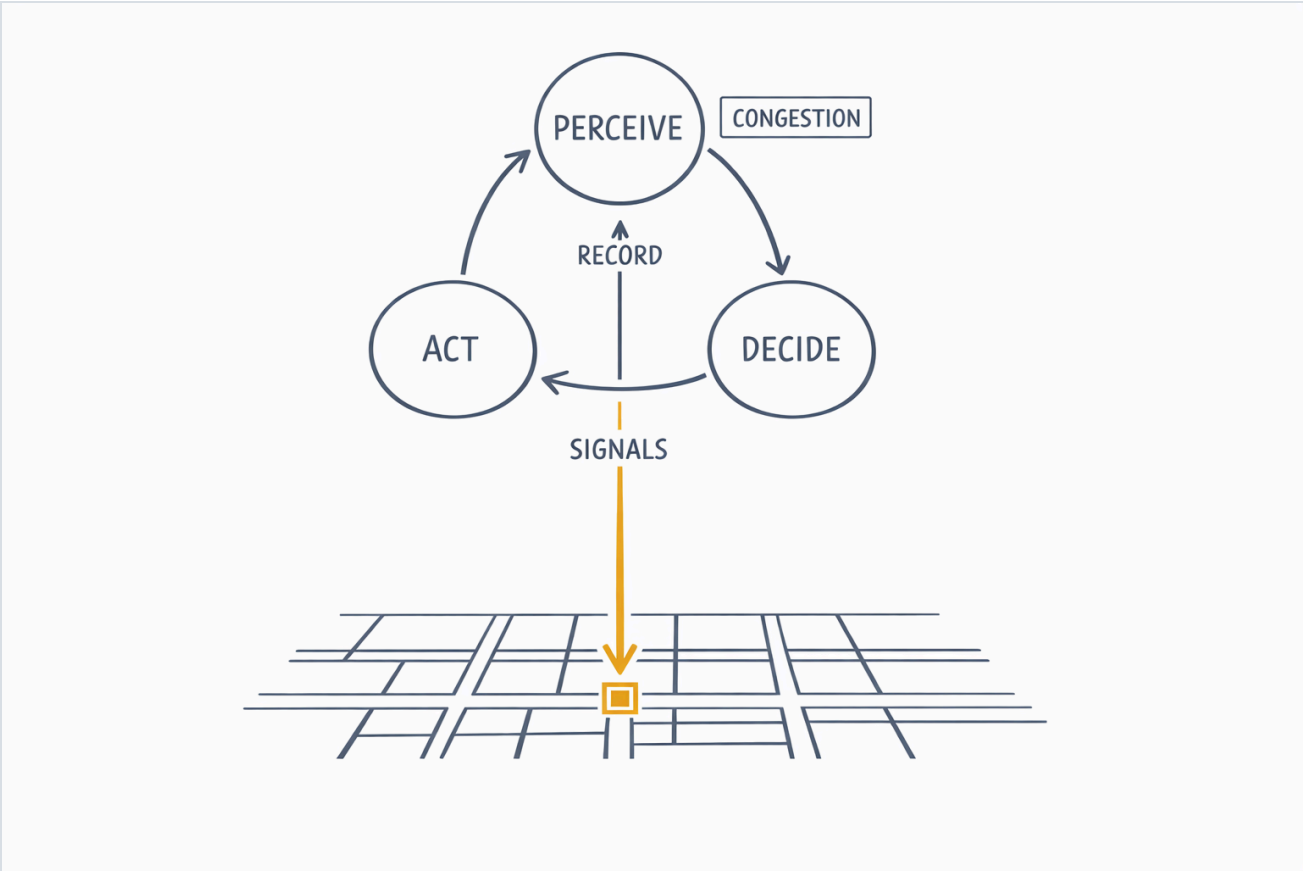


FIGURE 7 – THE CLOSED LOOP ON A LIVE CITY: PERCEIVE FROM THE RECORD, DECIDE, ACT ON SIGNALS AND ROUTES.

Surface	Detail
Sync tools	<code>query_state · describe_scenario · set_signal_phase · reroute_vehicle · set_edge_speed · close_lane · open_lane</code>
Durable tasks	<code>run_batch</code> (live sim — fails closed on interruption) · <code>generate_network</code> · <code>compute_routes</code> · <code>optimize_signals</code> (real SUMO programs, resumable)
Events	<code>sumo://congestion</code> is subscribable — a jam forming is a wake the moment it happens.
Outputs	Successful task outputs upload through a task-bound write capability to the shared artifact plane.



10.1 The Loop, Measured

The exact detach → sleep → wake path: the agent calls a task-augmented tool, receives a durable handle immediately, persists the descriptor, ends the episode, and sleeps. The module runs on; completion pushes a wake, and a fresh episode assembles with the result.

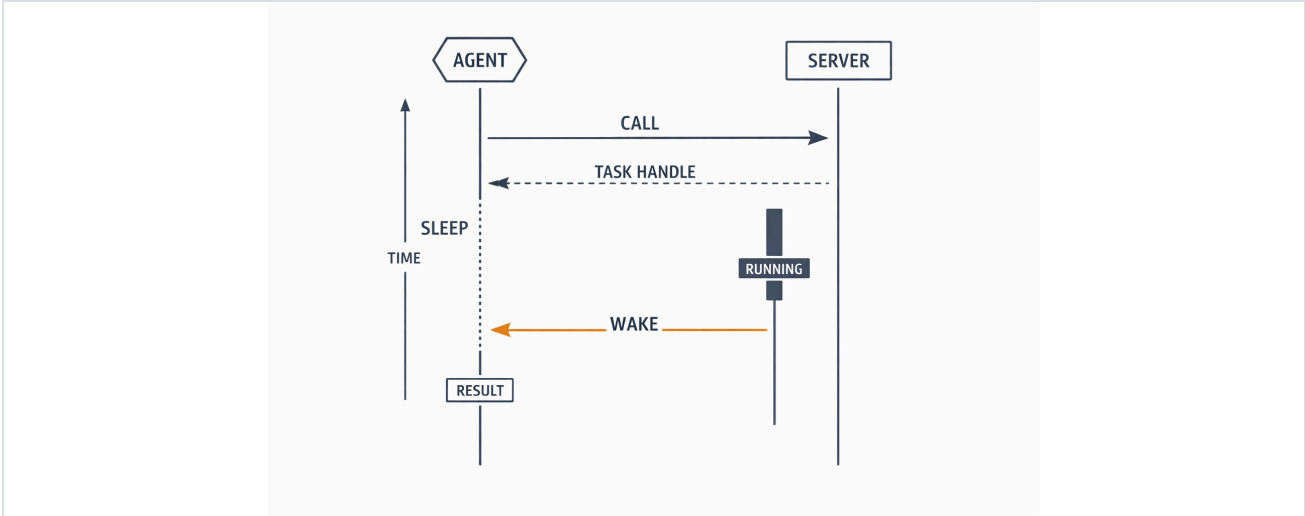


FIGURE 8 – DETACH → SLEEP → WAKE: ZERO CONNECTIONS HELD, ZERO EPISODES BURNED WAITING.

Property	Measured behavior
Capture throughput	~225,000 messages per second, lossless, on the persist-first path.
Crash recovery	kill -9 mid-write leaves a decodable segment; restart writes a sibling and never overwrites; startup reconciles leftovers into catalog records.
Wake latency	With polling pinned at 60 s and the heartbeat at 600 s, the agent slept a ~15 s task in full and woke within one second of completion — push, arithmetically.
Process death	Task descriptors and watcher leases rehydrate on any boot; kill and restart resumes the same tasks with memory intact.

Methodology

- **Smoke harness.** End-to-end scenarios with real process orchestration: fake identity provider, provider, and upstream services; a real enterprise IdP realm; live agent sleep/wake; and the full traffic world in a disposable project.
- **Conformance.** A generic MCP client exercises the full protocol surface, plus auth-metadata discovery and token tooling; continuous integration runs the full suite, the console build, the IdP integration path, and the deployment contract checks on every change.



11. Deployment Considerations

Deployment is a spectrum, and cognition is constant across it. The same platform runs at the operation's edge on one host, across a cluster, sealed inside an air gap, or hybrid — an edge installation that reaches remote capability when links allow. Wherever it lands, it carries the full stack: one agent or a team of them, every capability module, and the operational intelligence to run the mission — the console, the record, and the audit trail.

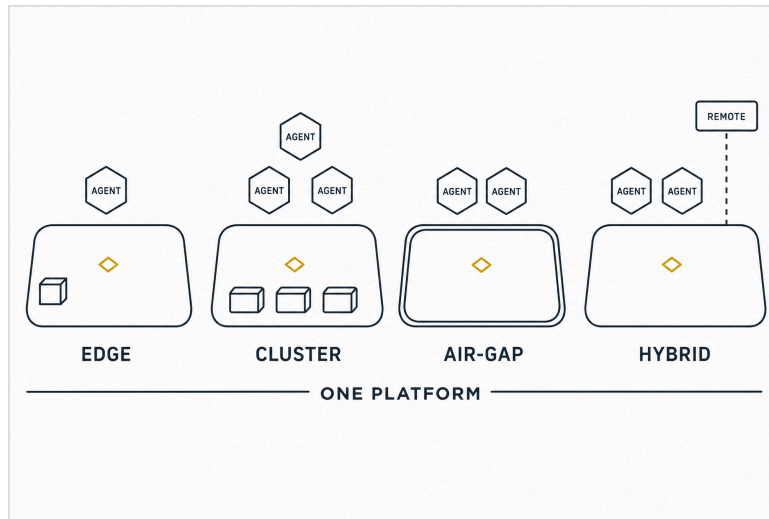


FIGURE 9 — SAME PLATFORM, SAME COGNITION — EDGE, CLUSTER, AIR-GAP, OR HYBRID; ONLY HYBRID REACHES OUT, AND ON THE OPERATOR'S TERMS.

Form Shape

Edge One host at the operation: a single command brings up the full stack behind a loopback edge. Public exposure is the operator's ingress decision.

Cluster The Helm form: one platform-store StatefulSet, default-deny network policy, optional service-mesh mTLS, separate migration and runtime credentials, operator-supplied telemetry and SIEM wiring.

Air-gap The verified offline bundle: pinned images, installation material, configuration schemas, checksums, and SBOMs. Installation and verification complete entirely inside the boundary.

Hybrid An edge installation that registers remote MCP endpoints and provider-backed modules. The reach is governed like everything else — same policy, same audit, same task contract.

12. Conclusion

The Autonomy Harness closes the loop that persistent autonomy requires. Agents reason from a world model that outlives every context window; they act through a gateway that enforces the operator's policy outside the model's own reasoning; their long work is durable, with recovery semantics declared per operation; and everything the operation senses and decides is persisted first and preserved as evidence. A live traffic world exercises the whole loop — perceive, decide, act, wake — and the same platform deploys unchanged from a single edge host to an air-gapped cluster.

The contracts are the foundation for what comes next: teams of agents coordinating over the same world model, richer capability modules registered from the ecosystem, and fleet-level guarantees expressed over mission outcomes. Each grows on the same spine this paper describes — the world model, the envelope, the durable task, and the record.

[THESIS, RESTATED]

Give agents a world model instead of a chat history. Govern them from outside their own reasoning. Make every piece of long work durable. Persist every signal first. And let one operator own the whole thing, anywhere the mission runs.



Appendix A: Component Reference

Component	Role	What it is
ingress	ours	The single public door, owned by the operator. One place to defend and observe.
mcp-gateway	ours	The front desk for tools: authenticates the caller, checks policy, projects every module's tools into one flat menu, writes the audit trail.
console	ours	The cockpit: the operations UI plus a session backend that owns browser login, encrypted sessions, and CSRF enforcement.
agent-kernel	ours	The runtime for long-lived agents: short episodes, durable memory, wake on a result, a timer, or a message.
world model	concept	What an agent knows: everything the operation senses, remembers, and decides, fused into one durable, queryable model of reality.
platform store	OSS	SurrealDB — the platform's memory: identity, policy, tasks, artifacts, recordings, agents, audit, and the transactional outbox.
task runtime	ours	The ledger for long work: atomic leases, transitions, results, and declared recovery per operation.
ingest socket	OSS	Where every producer pushes: one gRPC endpoint (the Rerun proxy), rebroadcast live to anyone reading.
persist-first writer	ours	Embeds the ingest socket and fsyncs every message to disk before anything else — the write happens first.
.rrd segment	format	The record on disk: one growing file per dataset / day / recording, readable even mid-write.
catalog	ours	The card catalog: typed installation identity for every recording and segment; startup reconciles crash leftovers.
recordings module	ours	The reading room: authorized discovery, latest-at / range / dataframe queries, subscriptions, and sealing to an artifact.
SUMO / TraCI	OSS	The traffic simulator and its control socket — a live world producing positions and states to record and reason about.
sumo-mcp	ours	The bridge: the one owner of the simulator's control socket, pushing the world into the record and exposing governed control.
MCP	protocol	The tool language every module speaks — which is why one gateway can front them all with one set of rules.
durable task	concept	A tool call you walk away from: the handle returns immediately; the row, lease, and result live in the platform store.
resource · subscribe	concept	A condition that wakes you: subscribing to a resource means the agent is notified the moment it changes.

